Practice Problem Set: Binary Search Trees and Recursion in C

General Instructions

- Use the C/C++ programming language.
- Use recursion wherever specifically mentioned.
- For questions marked **"Write only the function,"** do not write the complete program.
- Clearly define the structure of a tree node where necessary.
- Assume that all values in a Binary Search Tree are distinct unless stated otherwise.

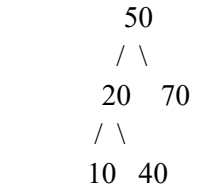
Part A: Binary Search Tree and Binary Tree Problems

1. Delete All Leaf Nodes from a BST

Write a function that deletes all the leaf nodes from a Binary Search Tree. The function should also print the values of the deleted nodes.

Write only the function.

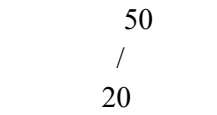
For example, consider the following BST:



Output:

Deleted nodes: 70, 10, 40

After deletion, the BST will be:



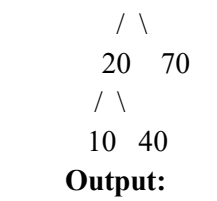
---

2. Count the Leaf Nodes of a BST

Write a function that counts the total number of leaf nodes in a Binary Search Tree.

Write only the function.

For example:



Output:

Count of leaf nodes: 3

---

3. Construct a Binary Tree from Traversals

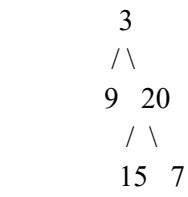
Write a C program that constructs a binary tree from its given preorder and inorder traversals. After constructing the tree, find and display its height.

Input:

Preorder: 3 9 20 15 7

Inorder: 9 3 15 20 7

Constructed tree:



Output:

Height of the tree: 3

Assume that all node values are distinct.

---

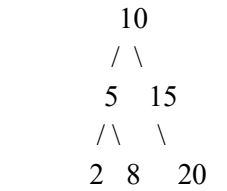
4. Maximum Downward Path Sum in a BST

Write a C program that calculates the maximum downward path sum in a Binary Search Tree.

A downward path:

- May start at any node.
- May end at any descendant of that node.
- Can move only from a parent node to one of its child nodes.

For example:



Possible downward paths include:

10 → 15 → 20

5 → 8

15 → 20

20

Output:

Maximum downward path sum: 45

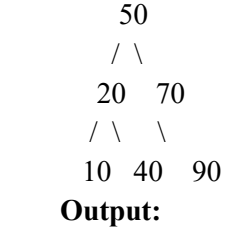
---

5. Count Nodes with Two Children

Write a recursive function that counts the total number of nodes in a BST that have both a left child and a right child.

Write only the function.

For example:



Output:

Number of nodes with two children: 2

---

6. Find the Second Largest Element in a BST

Write a function that finds the second largest element in a Binary Search Tree without storing the tree elements in an array.

The function should work for the following cases:

- The largest node has a left subtree.
- The largest node does not have a left subtree.
- The BST contains fewer than two nodes.

For example:

```
      50
     /  \
    30    70
       /  \
      60   90
```

Output:

Second largest element: 70

---

7. Print All Nodes Within a Given Range

Write a recursive function that prints all values of a BST that lie within a given range [low, high].
The values should be printed in ascending order.

Write only the function.

For example:

```
      50
     /  \
    30    70
   / \    / \
  20 40  60 80
```

For:

low = 35

high = 75

Output:

40 50 60 70

The function should use the BST property to avoid visiting unnecessary nodes.

---

8. Check Whether a Binary Tree Is a BST

Write a recursive function that determines whether a given binary tree satisfies the properties of a Binary Search Tree.

For every node:

- All values in its left subtree must be smaller.
- All values in its right subtree must be greater.
- Both subtrees must also be valid BSTs.

Write only the function.

The function should return:

1 if the tree is a BST

0 otherwise

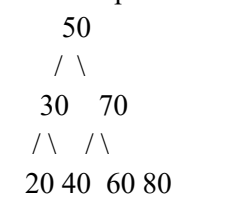
---

9. Find the Lowest Common Ancestor in a BST

Write a function that finds the Lowest Common Ancestor, or LCA, of two given values in a Binary Search Tree.

The LCA of two nodes is the lowest node in the tree that has both nodes as descendants. A node may be considered a descendant of itself.

For example:



For nodes 20 and 40:

Lowest Common Ancestor: 30

For nodes 20 and 80:

Lowest Common Ancestor: 50

---

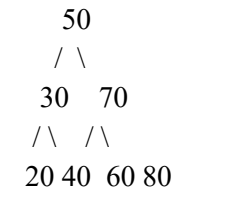
10. Delete All Nodes Outside a Given Range

Write a recursive function that removes all nodes from a BST whose values are outside a given range [low, high].

The remaining tree must still satisfy the properties of a BST.

Write only the function.

For example:



For:

low = 35

high = 70

The resulting BST will contain:

40 50 60 70

Part B: Recursion Problems

1. Reverse a String Recursively

Write a C program that uses a recursive function to reverse a given string in place.

Requirements:

- The original string must be modified.
- Do not use an additional character array or buffer for the reversed string.
- The recursive function may use indices or pointers.

Sample input:

hello

Sample output:

olleh

---

2. Delete All Occurrences of a Character

Suppose you are given a string S and a specified character. Write a recursive function that deletes all occurrences of the specified character from the string.

Write only the recursive function.

Input:

Input string: one$two#three$

Specified character: o

Output:

ne$tw#three$

The function must modify the original string.

---

3. Count the Occurrences of a Character

Write a recursive function that counts how many times a specified character occurs in a given string.

Write only the function.

Example:

Input string: programming

Specified character: g

Output:

Number of occurrences: 2

---

4. Find the Sum of Digits

Write a recursive function that calculates the sum of the digits of a positive integer.

Write only the function.

Example:

Input: 5834

Output: 20

Because:

5 + 8 + 3 + 4 = 20

---

5. Check Whether a String Is a Palindrome

Write a recursive function that checks whether a given string is a palindrome.

A palindrome reads the same from left to right and from right to left.

Write only the function.
Example 1:
Input: level
Output: Palindrome
Example 2:
Input: hello
Output: Not a palindrome
The function should not create a reversed copy of the string.

---

6. Find the Greatest Common Divisor
Write a recursive function that calculates the Greatest Common Divisor, or GCD, of two positive integers using Euclid's algorithm.
Write only the function.
Euclid's recursive definition is:
GCD(a, b) = GCD(b, a % b)
The recursion stops when b becomes zero.
Example:
Input: 48 18
Output: 6

---

7. Convert a Decimal Number to Binary
Write a recursive function that displays the binary representation of a positive decimal integer.
Do not use an array to store the binary digits.
Write only the recursive function.
Example:
Input: 13
Output: 1101

---

8. Find the Largest Element in an Array
Write a recursive function that finds the largest element in an integer array.
Write only the function.
Example:
Array: 12 7 35 19 24
Output:
Largest element: 35
The function must not use any loop.

---

9. Move All Occurrences of a Character to the End
Write a recursive function that moves every occurrence of a specified character to the end of a string while maintaining the relative order of the remaining characters.
Example:
Input string: axbxacxx
Specified character: x
Output:

abacxxxx
The function may modify the original string but must not use another complete string to store the result.

---

10. Generate All Subsequences of a String
Write a recursive C program that prints all possible subsequences of a given string.
A subsequence is formed by including or excluding each character while preserving the original order.
Example:
Input: abc
Possible output:
abc
ab
ac
a
bc
b
c
Empty string
The order of the generated subsequences may vary.
The function must use recursion to make the following two decisions for every character:

1. Include the character.
2. Exclude the character.